

Relazione del Progetto di Sistemi Operativi

Strumento Client-Server per la computazione di impronte SHA-256

Ariana Baci
Corso di Laurea in Informatica
Università degli Studi di Verona

5 agosto 2026

1 Obiettivi e specifiche implementate

L'obiettivo del progetto è consentire a diversi client, anche simultanei, di inviare un file e ricevere l'impronta SHA-256 relativa al contenuto.

Per la realizzazione del sistema ho scelto di utilizzare delle FIFO per la comunicazione tra client e server e dei thread concorrenti per la gestione di richieste multiple in parallelo.

Descrizione delle funzionalità implementate

1. (Punti 1 e 2) **Server e client che inviano richieste e ricevono risposte:** il server apre una FIFO in lettura, che ciascun client apre in modalità scrittura per inviare la propria richiesta; il client apre una FIFO in lettura, che il thread dedicato alla sua richiesta apre in scrittura per fornire l'impronta SHA al termine della computazione.
2. Il server istanzia thread distinti per elaborare richieste multiple in modo concorrente.
3. Le richieste, prima di essere assegnate a un thread, vengono inserite in una lista per poter essere prelevate in ordine di dimensione del file.
4. Il numero massimo di thread in esecuzione è fissato da una variabile (`MAX_THREADS`, in `common.h`).
5. Implementazione del caching in memoria delle coppie percorso-hash già servite: una lista cache contiene le coppie percorso-hash già servite (fino a un limite stabilito dalla variabile `MAX_CACHE_ENTRIES`); ogni richiesta viene processata usando l'algoritmo per il calcolo dell'impronta SHA solo se nella cache non è presente la entry corrispondente al percorso del file, altrimenti si restituisce al client direttamente il valore associato.

6. Richieste multiple simultanee per un dato percorso vengono gestite processando una sola richiesta e attendendo il risultato per le restanti: quando un thread deve calcolare un'impronta, prima di iniziare la computazione inserisce in cache il percorso del file, associandolo a un valore nullo e marcandolo come `ENTRY_IN_PROGRESS`, poi sblocca la cache. In questo modo gli altri thread sono al corrente del fatto che l'impronta per quel file non è ancora in cache ma sta già venendo calcolata da un altro processo.
7. Implementazione di un "thread pool": il server crea `MAX_THREADS` thread distinti senza aspettare di ricevere richieste. I thread girano quindi continuamente, processando richieste e verificando la presenza di nuove richieste da servire. In questo modo si evita il sovraccarico dovuto alla creazione e distruzione di thread per ogni richiesta.
8. Implementazione di un meccanismo di interrogazione della cache delle richieste già processate: i thread interrogano la cache tramite la funzione `cache_get_or_compute`, che a sua volta chiama `find_entry`, la quale scorre le richieste processate cercando la chiave (nome del file) passata come parametro.

2 Struttura del codice

Il codice ha una struttura modulare ed è suddiviso in più file, ciascuno dei quali contiene funzioni e strutture dati specifiche per una determinata funzionalità. I principali:

1. `src/server.c` : contiene il ciclo principale che ascolta le richieste dei client dalla FIFO e le inoltra ai thread
2. `src/client.c` : invia richieste tramite FIFO, sotto forma di file, attende e legge la risposta
3. `src/digest.c` : algoritmo che calcolo dell'hash SHA256 di un file
4. `src/cache.c` : cache delle coppie (percorso, hash) e gestione richieste concorrenti sullo stesso file
5. `src/threadpool.c` : coda delle richieste pendenti e pool di thread worker
6. `src/errExit.c` : gestione centralizzata degli errori fatali
7. `inc/*.h` : dichiarazioni e strutture condivise tra i file sopra

3 Flusso di esecuzione

Il flusso di esecuzione parte dal lancio del client, a cui viene passato come parametro il nome del file di cui si vuole calcolare l'impronta SHA. Il client comunica la richiesta al server tramite FIFO. Il server, sempre in ascolto, legge quanto

scritto nella FIFO dai vari client e inoltra le richieste al threadpool. Ciascun thread, quando riceve una richiesta, consulta la cache, che contiene coppie percorso file - impronta SHA. Se trova il percorso del file richiesto, ne ritorna direttamente il valore di impronta associato, eventualmente attendendo se tale valore non è ancora valido ma un altro thread lo sta calcolando; se non lo trova, chiama la funzione `digest`, che calcola l'impronta a partire dal contenuto del file, e successivamente, nel più dei casi, salva in cache i dati della nuova richiesta servita. Ogni thread infine invia, sempre tramite FIFO, il risultato al client, che lo legge e lo stampa.

4 Analisi dei moduli e scelte implementative

L'organizzazione in moduli del progetto è stata decisa ed elaborata implementando le funzionalità che richiedevano la cache, per rendere il codice più gestibile e comprensibile.

4.1 Server

Il server crea la fifo con permessi di lettura e scrittura a owner, group e other, definiti da `FIFO_PERMISSIONS` in `common.h`, come anche il percorso (`'/tmp/server_fifo'`), noto anche al client. Chiama una funzione per inizializzare i thread e apre la propria FIFO in lettura e resta perennemente in ascolto di richieste tramite un ciclo `while(1)`. A ogni richiesta ricevuta, calcola la dimensione del file e la inserisce in una struttura dati apposita, insieme alle informazioni ricevute dal client, ovvero percorso per la sua FIFO e nome del file. Il server a questo punto non fa che inoltrare la richiesta così pre processata al threadpool.

Per quanto riguarda le difficoltà riscontrate, questo è stato il file che ha subito più modifiche. Con l'aumentare delle funzionalità, essendo partita da una base senza cache e threadpool, ho infatti modificato diversi aspetti (ad esempio, all'inizio le richieste erano processate in ordine di arrivo e non di dimensione del file, e in assenza di thread pool venivano mandate direttamente a un thread creato sul momento, senza essere messe in coda e confrontate), e ho deciso di snellire il più possibile questa sezione demandando diverse funzioni ad altri moduli.

4.2 Client

Ciascun client si assicura di aver ricevuto come parametro il nome del file da processare e nient'altro. Crea un proprio percorso univoco per la FIFO tramite `getpid()` (altrimenti rischierebbe di ricevere la risposta indirizzata a qualche altro processo), e apre la FIFO del server, di cui conosce il path in quanto dichiarato nel codice, in modalità scrittura. Invia al server il nome del file ricevuto da riga di comando e il percorso alla propria FIFO (aperta in lettura), resta quindi in attesa della risposta, che infine stampa.

4.3 Threadpool

Nel modulo è presente la funzione `threadpool_submit`, chiamata dal server per l'inoltro delle richieste, che salva le richieste in una coda, fino a un limite massimo. Ciascun thread esegue continuamente un ciclo nel quale, se la coda delle richieste è vuota, attende, altrimenti sceglie quella col file associato di dimensione più piccola e la manda alla cache perchè ne restituisca il risultato. Terminata la computazione svolta dalla cache, apre la fifo del client in scrittura e manda la risposta. I thread vengono creati con `pthread_create` e immediatamente distaccati tramite `pthread_detach`: quindi il server non attende mai la terminazione dei thread, e le risorse di ciascun thread vengono liberate automaticamente al termine della sua esecuzione. Questa scelta è coerente con il modello a thread pool adottato: i thread non terminano mai, quindi una `pthread_join` non avrebbe comunque senso, e `pthread_detach` serve principalmente a evitare memory leak sulle strutture interne del thread stesso. Ho scelto di passare la variabile "hash" come parametro così che le modifiche effettuate dalla cache siano direttamente visibili anche alla funzione chiamante. Ho pensato di non mantenere una lista ordiata per le richieste ma far cercare a ciascun thread quella più piccola: la complessità è equivalente, infatti in entrambi i casi dovrei fare una scansione lineare per cercare in un caso, per inserire in maniera ordinata nell'altro, per ogni richiesta.

4.4 Cache

La cache viene chiamata dai thread tramite `cache_get_or_compute`. Usa una coda condivisa protetta da mutex a cui ogni processo accede mutualmente per verificare la presenza di una entry e inserirne una nuova, mentre durante l'operazione di calcolo dell'impronta SHA viene sbloccata così che gli altri thread non debbano attendere durante la chiamata alla funzione che processa effettivamente il contenuto del file. Per segnalare che una entry è in fase di computazione ho aggiunto la variabile `entry_state_t` che ne dichiara lo stato, che può essere `ENTRY_IN_PROGRESS` oppure `ENTRY_READY`. Nel caso in cui in cache ci sia già il numero massimo di record, viene cercata randomicamente una entry che non sia dichiarata come `ENTRY_IN_PROGRESS` e sovrascritta. Se non viene trovata nessuna entry in stato `ENTRY_READY` entro un numero di tentativi pari al numero massimo di record, l'impronta viene calcolata e restituita al client senza essere salvata: la scelta è stata fatta per non creare computazioni complesse che richiedessero più tempo di una scansione lineare e allo stesso tempo evitare di preferire sempre la sostituzione delle prime entry della cache.

4.5 Digest

Il modulo contiene la funzione `digest`, che calcola l'impronta SHA256 del file passato come parametro. Il codice è sostanzialmente analogo a quello fornito dal docente.

4.6 common.h

Il file contiene le dichiarazioni delle principali strutture dati e costanti utilizzate in tutto il progetto, come ad esempio il numero massimo di thread, il percorso della FIFO del server e i permessi per la creazione della FIFO, al fine di rendere più leggibile il codice e centralizzare le modifiche in caso di necessità.

5 Test svolti

5.1 Ordine di esecuzione delle richieste

Per verificare che le richieste vengano eseguite in ordine di dimensione del file, ho impostato `/textttMAX_THREADS` a 1, in modo da consentire che le richieste si accumulino nella cache (altrimenti sarebbero eseguite in ordine di arrivo), e ho poi usato lo script `testOrdineEsecuzione.sh`, che crea 5 file di diverse dimensioni e lancia 5 client in parallelo, uno per ciascun file. Ho verificato che le risposte ricevute dai client fossero in ordine di dimensione del file, indipendentemente dall'ordine di lancio dei client stessi.

5.2 Utilizzo dei dati presenti in cache

Per verificare che i processi non calcolino impronte già presenti in cache, pronte o fase di computazione, ho reimpostato `/textttMAX_THREADS` a un valore ragionevole, 1 e usato lo script in `testCaching.sh`, che crea un file sufficientemente grande e lancia 15 client in parallelo con la stessa richiesta. Ne è risultato che i processi, dopo il primo, che consultano la cache, trovano la entry in stato `ENTRY_IN_PROGRESS` e stampano "waiting for another thread that is already computing hash for file ...", una volta che lo stato della entry passa a `ENTRY_READY` ciascun thread (eccetto il primo), usa il valore salvato e stampa "using cached hash for file ..."